UKAPI.io

MVP Product Report

Product scope, launch catalogue, pricing, UX and delivery plan

Prepared 23 September 2026

Primary stack: Laravel 13 / PHP 8.3+

Simple UK data. One API.

CONFIDENTIAL WORKING DOCUMENT

Version: 1.0 **Date:** 23 September 2026 **Primary stack:** Laravel 13 / PHP 8.3+ **Product type:** Low-cost UK data API platform

Executive summary

UKAPI.io is a developer-first service that exposes many small, useful UK-focused data APIs behind one account, one API key, one documentation site, one billing relationship and one predictable response format. The product does not attempt to become the authoritative owner of every underlying dataset. Instead, it acts as a reliable aggregation and normalisation layer over public/open data sources and selected first-party APIs.

The MVP should be deliberately narrow operationally while appearing broad to the customer. A single Laravel application can provide the website, authentication, API-key management, metering, billing, documentation metadata and the API gateway. Each upstream dataset is isolated behind a provider adapter so public endpoint contracts remain stable even if an upstream service changes.

The first release should focus on endpoints that are cheap to operate, broadly useful, legally reusable and straightforward to normalise. Recommended launch families are: postcodes and location, companies, dates and bank holidays, councils, food hygiene, crime summaries, flood information and selected planning constraints. Higher-risk or high-cost sources such as vehicle data, address autocomplete from commercial PAF data, credit information, identity data or anything requiring expensive licensing should remain outside the MVP.

A sensible commercial position is “simple UK data for developers” rather than “a premium data provider”. The free plan should be genuinely useful, while paid plans primarily buy higher rate limits, larger monthly quotas, priority caching and commercial support.

1. Product vision

1.1 Product statement

UKAPI.io gives developers one simple API for useful UK data.

Instead of separately learning Companies House, Police.uk, GOV.UK, local authority datasets, planning datasets and postcode services, a developer can authenticate once and use a consistent URL structure and JSON envelope.

1.2 Problem

UK public data is plentiful but fragmented. Common integration friction includes:

- different authentication methods;
- different rate-limit behaviour;
- inconsistent field naming and date formats;
- differing error structures;
- upstream APIs that expose more fields than most applications need;
- postcode-based use cases that actually require coordinate conversion first;
- inconsistent geographic coverage across England, Wales, Scotland and Northern Ireland;
- varying stability, beta status and licensing requirements;

- duplicated caching and resilience work in every consuming application.

UKAPI.io removes that integration tax.

1.3 Target users

The primary MVP users are solo developers, agencies, small SaaS products, internal business tools and prototypes. Secondary users include students, data journalists and small organisations that need a few reliable lookups but do not want enterprise contracts.

The MVP should not target regulated financial decisioning, KYC/AML, insurance underwriting, credit scoring, emergency-response systems or other use cases where the platform would need additional contractual guarantees and compliance controls.

2. MVP goals and non-goals

2.1 Goals

1. Provide one API key for multiple UK datasets.
2. Offer predictable JSON conventions across every endpoint.
3. Cache public upstream data aggressively enough to control cost and improve reliability.
4. Meter usage per API key, endpoint family and day/month.
5. Provide a useful free tier and inexpensive self-service paid plans.
6. Make documentation excellent enough that a developer can make a successful request in less than five minutes.
7. Keep source-specific logic isolated so sources can be replaced without breaking public contracts.
8. Make operational health visible to the owner through internal dashboards and alerts.
9. Launch with enough endpoint families that the “many mini APIs” proposition is immediately credible.

2.2 Non-goals

The MVP will not:

- guarantee real-time freshness for every source;
- ingest entire national datasets into a bespoke warehouse unless needed for reliability;
- provide an SLA beyond best-effort service on the free and low-cost tiers;
- offer user-submitted data;
- expose personally sensitive datasets beyond information already made publicly available by authoritative sources;
- proxy arbitrary upstream URLs;
- offer GraphQL initially;
- provide write-through access to government services;
- build mobile applications;
- support bespoke enterprise contracts at launch.

3. Brand and positioning

3.1 Name

UKAPI.io

3.2 Suggested positioning

Primary tagline: **Simple UK data. One API.**

Alternative homepage line: **Postcodes, companies, councils, crime, holidays, planning and more through one clean API.**

3.3 Brand personality

The visual identity should feel like infrastructure rather than government. Use a clean developer-tool aesthetic, restrained colour palette, clear monospace examples and minimal marketing language. Avoid visual imitation of GOV.UK, which could imply official status.

Every relevant page should include a clear statement such as: **UKAPI.io is an independent developer service and is not affiliated with or endorsed by the UK Government.**

4. MVP endpoint catalogue

The endpoint catalogue should be versioned under /v1. Endpoint naming should use plural nouns where the resource is naturally plural and action-like suffixes only for derived utilities.

4.1 Postcodes and location

Recommended endpoints:

```
GET /v1/postcodes/{postcode}
GET /v1/postcodes/{postcode}/validate
GET /v1/postcodes/{postcode}/nearby
GET /v1/coordinates/{latitude}/{longitude}/postcode
GET /v1/outcodes/{outcode}
```

Core response fields should include canonical postcode, country, region where applicable, coordinates, local authority name/code, parliamentary constituency where available and selected statistical geography codes.

The adapter can initially use Postcodes.io, which provides postcode lookup and reverse geocoding without authentication. The public UKAPI.io contract should remain source-agnostic so a future ONS-backed/self-hosted implementation can replace it.

4.2 Councils

Recommended endpoints:

```
GET /v1/councils/postcode/{postcode}
GET /v1/councils/{code}
```

For MVP, council lookup can derive primarily from postcode administrative geography plus a maintained council metadata table containing human-friendly names, websites, country, type and identifiers.

A council endpoint is valuable because many future mini APIs can use council identity as a common join key.

4.3 Companies

Recommended endpoints:

```
GET /v1/companies/{companyNumber}
GET /v1/companies/{companyNumber}/officers
GET /v1/companies/{companyNumber}/filings
GET /v1/companies/search?q={query}
GET /v1/sic/{code}
GET /v1/sic/search?q={query}
```

Companies House provides live public company information through its REST API and requires API credentials. UKAPI.io can hide that integration detail while returning a simplified stable schema.

The MVP should avoid promising that every filing document itself can be redistributed. Start with metadata exposed by the public data API and review terms/licensing before adding document proxying.

4.4 Dates and business utilities

Recommended endpoints:

```
GET /v1/bank-holidays
GET /v1/bank-holidays/{division}
GET /v1/working-days?from=YYYY-MM-DD&to=YYYY-MM-DD&division=england-and-wales
GET /v1/tax-years/current
GET /v1/tax-years/{year}
GET /v1/vat/calculate?amount=100&rate=20
GET /v1/vat/remove?amount=120&rate=20
```

These are exceptionally cheap endpoints and make useful “glue” utilities. Bank holidays should come from the official GOV.UK JSON source. Working-day and VAT endpoints are derived utilities implemented directly by UKAPI.io.

Do not market tax endpoints as tax advice. For MVP, tax-year utilities should deal with calendar boundaries and identifiers, not attempt to calculate individual tax liabilities.

4.5 Food hygiene

Recommended endpoints:

```
GET /v1/food-establishments/{id}
GET /v1/food-establishments/search?name=&postcode=&rating=
GET /v1/food-establishments/nearby?postcode={postcode}&radius=2
```

The Food Standards Agency offers free programmatic access to FHRS/FHIS data across the UK with no registration requirement at the time of writing. UKAPI.io should normalise the differences between rating schemes instead of pretending every nation uses the same score system.

4.6 Crime

Recommended endpoints:

```
GET /v1/crime/nearby?postcode={postcode}&month=YYYY-MM
GET /v1/crime/summary?postcode={postcode}&month=YYYY-MM
GET /v1/crime/categories
```

The Police.uk street-level endpoint works from coordinates and returns approximate/anonymised crime locations. UKAPI.io can make postcode searches convenient by resolving the postcode to coordinates first.

Coverage caveats must be exposed in metadata, particularly for Scotland where Police.uk does not provide a complete national picture.

4.7 Flood information

Recommended endpoints:

```
GET /v1/flood/warnings
GET /v1/flood/nearby?postcode={postcode}
GET /v1/flood/stations/nearby?postcode={postcode}
GET /v1/flood/stations/{id}/readings
```

The Environment Agency provides near-real-time flood warnings, alerts, station information and readings for England. Responses must clearly identify geographic scope and source timestamps.

4.8 Planning constraints

Recommended MVP endpoints:

```
GET /v1/planning/constraints?postcode={postcode}
GET /v1/planning/constraints?lat={lat}&lng={lng}
```

Start with a small curated group of high-value datasets such as conservation areas, listed buildings, green belt, Article 4 directions and flood-risk zones. The official Planning Data API exposes more than 100 planning/housing datasets for England, but its documentation currently describes the platform/API as beta. UKAPI.io should therefore cache and isolate it carefully and return source status in metadata.

Planning coverage should be labelled **England only** until equivalent, dependable sources are intentionally integrated for the other nations.

5. Standard API contract

5.1 Base URL

```
https://api.ukapi.io/v1
```

5.2 Authentication

Customer requests use an API key, preferably in an HTTP header:

```
Authorization: Bearer uk_live_xxxxxxxxxxxxxxxxx
```

Keys should have a visible prefix (uk_live_, uk_test_) and only the hash should be stored after creation.

5.3 Success envelope

```
{
 "data": {},
 "meta": {
  "request_id": "req_01...",
  "cached": true,
  "source": "companies_house",
  "source_updated_at": "2026-09-23T12:30:00Z"
 }
}
```

For list endpoints:

```
{
 "data": [],
 "meta": {
  "request_id": "req_01...",
  "pagination": {
   "page": 1,
   "per_page": 25,
   "total": 113
  }
 }
}
```

5.4 Error envelope

```
{
 "error": {
  "code": "postcode_not_found",
  "message": "The postcode could not be found.",
  "status": 404,
  "request_id": "req_01..."
 }
}
```

Never leak upstream HTML errors or source-specific exception messages.

5.5 HTTP conventions

Use standard status codes: 200, 400, 401, 403, 404, 409, 422, 429, 502 and 503 where appropriate. A source outage should generally map to 503 upstream_unavailable, while malformed upstream data may map to 502 upstream_invalid_response.

6. Account and dashboard MVP

The customer dashboard should include:

- register, login, password reset and email verification;
- API key creation, naming, last-used timestamp and revocation;

- current plan and billing management;
- monthly request usage and quota progress;
- endpoint-family usage breakdown;
- recent API requests with status code and latency, but without logging sensitive query content unnecessarily;
- documentation links and quick-start code snippets;
- account settings;
- service status link.

API key values are shown only once at creation. Users can create multiple keys so local development, production and individual applications remain separated.

7. Pricing model

Keep pricing extremely simple at launch.

Plan	Suggested price	Included requests/month	Intended user
Free	£0	5,000	evaluation, hobby projects
Hobby	£4.99	50,000	small applications
Pro	£12.99	250,000	production SaaS / agencies
Scale	£29.99	1,000,000	higher-volume small businesses

Initial overage billing should be avoided. Hard or soft quota limits are easier to understand and reduce billing disputes. Allow users to upgrade instantly when a quota is reached.

Certain expensive endpoint families can later consume weighted credits rather than one request each, but MVP endpoints should ideally all count as one request to preserve simplicity.

8. Recommended Laravel product stack

The MVP can remain a single Laravel application.

- **Framework:** Laravel 13
- **Runtime:** PHP 8.3+
- **Database:** MySQL 8.x or compatible managed MySQL
- **Cache / queues / rate limiting:** Redis
- **Queue monitoring:** Laravel Horizon
- **Authentication:** Laravel's normal web authentication plus custom API-key middleware; Sanctum may be used for dashboard/browser API needs but customer server API keys should use a purpose-built model
- **Frontend:** Inertia + React + Tailwind CSS is a good fit for a polished single-repository dashboard; Blade/Livewire is also acceptable if preferred
- **Billing:** Stripe + Laravel Cashier
- **Email:** transactional provider such as Postmark, Resend, Mailgun or SES
- **Object storage:** S3-compatible storage only where needed

- **Observability:** Laravel logs plus an error tracker such as Sentry; uptime monitoring externally
- **Documentation:** generated OpenAPI document plus a polished documentation frontend

Laravel 13 is the current major release as of September 2026 and requires PHP 8.3 or newer.

9. Data-source strategy

Every external integration should implement a source adapter. Public controllers and domain services must not know how Companies House or Police.uk represent their raw fields.

Conceptually:

```
HTTP request
  -> endpoint service
    -> cache
      -> provider adapter
        -> upstream API
      -> normaliser
    -> public response DTO/resource
```

Each adapter owns:

- upstream authentication;
- HTTP request construction;
- upstream retry policy;
- timeout policy;
- response validation;
- source-specific error mapping;
- normalisation to internal DTOs;
- freshness/caching hints;
- attribution metadata.

This is one of the most important architectural decisions in the product.

10. Caching and freshness

Not every endpoint needs live upstream calls.

Suggested defaults:

Endpoint family	Initial cache TTL
postcode lookup	30 days
council metadata	30 days
company profile	15 minutes
company officers	1 hour
company filings	15 minutes
bank holidays	24 hours
food hygiene	6 hours
crime month data	24 hours
flood warnings	5 minutes
flood readings	5 minutes

Endpoint family	Initial cache TTL
planning constraints	24 hours

Cache TTLs are product defaults, not guarantees about source freshness. Return source_updated_at and, where useful, retrieved_at so consumers can reason about data age.

Use stale-while-revalidate for endpoints where a slightly old response is preferable to total failure.

11. Rate limiting and abuse controls

Enforce both burst and monthly controls.

Example starting points:

- Free: 5 requests/second burst, 5,000/month;
- Hobby: 10 requests/second, 50,000/month;
- Pro: 25 requests/second, 250,000/month;
- Scale: 50 requests/second, 1,000,000/month.

Rate limits should be applied per API key using Redis. Authentication failures and clearly abusive traffic should have separate stricter limits by IP.

Do not expose an unrestricted proxy to upstream services. Validate every path/query field and cap result sizes.

12. Documentation requirements

Documentation is a core MVP feature, not an afterthought.

Every endpoint page should show:

- method and path;
- description and geographic coverage;
- authentication requirement;
- parameters;
- example request;
- example success response;
- error codes;
- freshness expectations;
- source attribution;
- copy-paste examples for cURL, PHP/Laravel, JavaScript/TypeScript and C#;
- a “Try it” console using the logged-in user's test key.

Publish an OpenAPI 3.1 specification at a stable URL.

13. Security and privacy baseline

The MVP should include:

- hashed API keys at rest;

- encryption for upstream secrets;
- HTTPS only;
- CSRF protection on the web dashboard;
- strict validation of all route/query inputs;
- per-key and per-IP throttling;
- no raw upstream credentials in logs;
- redact API keys from request logs;
- role separation for admin access;
- two-factor authentication for platform administrators;
- audit logging for key creation/revocation and plan changes;
- dependency and vulnerability scanning in CI;
- regular database backups;
- retention limits for request logs.

Avoid retaining full IP addresses indefinitely unless there is a justified security/operational reason. Document retention and privacy behaviour clearly.

14. Admin MVP

A small internal admin area should provide:

- users and subscriptions;
- API keys and revocation;
- usage by endpoint/source;
- source health and last successful upstream request;
- recent 5xx errors;
- rate-limit events;
- cache hit percentage;
- ability to disable an endpoint family with a feature flag;
- service announcements;
- manual user quota adjustment;
- provider configuration health without exposing secret values.

Do not build a massive admin CMS. Operational visibility is the priority.

15. Reliability model

UKAPI.io depends on upstream providers, so graceful degradation is essential.

For each provider define:

- connect timeout;
- total request timeout;
- safe retry conditions;
- maximum attempts;
- cache fallback behaviour;
- circuit-breaker threshold;
- user-facing error code;

- health-check behaviour.

An upstream outage should not cascade into long PHP worker exhaustion. Fail quickly, use stale cache where safe and surface an honest 503 response when necessary.

16. Legal, licensing and attribution

Before launch, maintain a source register containing for each provider:

- source owner;
- source URL;
- licence/terms URL;
- permitted reuse summary;
- mandatory attribution wording;
- geographic scope;
- retention/caching constraints;
- redistribution constraints;
- owner/contact;
- date terms were last reviewed.

Open Government Licence data still carries attribution obligations. Individual APIs may have their own terms. “Publicly accessible” must never be treated as synonymous with “unrestricted commercial redistribution”.

Planning and property responses should clearly state that results are informational and are not a substitute for official searches or professional advice.

17. MVP UX/pages

Public website:

10. Homepage
11. API catalogue
12. Pricing
13. Documentation
14. Status
15. Terms
16. Privacy
17. Acceptable use
18. Data sources / attribution
19. Login / register

Authenticated dashboard:

20. Overview
21. API keys
22. Usage
23. Billing
24. Request logs
25. Account/security

Admin:

26. Overview
27. Users
28. Usage
29. Providers
30. Errors
31. Feature flags

18. Development phases

Phase 0 — foundation

- Laravel 13 application setup;
- environments and CI;
- authentication;
- database schema;
- Redis;
- provider-adapter interfaces;
- API-key model and middleware;
- response/error envelopes;
- request IDs;
- OpenAPI generation approach.

Phase 1 — first useful endpoints

Implement postcode, councils, bank holidays and VAT/working-day utilities. These give fast coverage with minimal source risk and validate the gateway, cache and documentation architecture.

Phase 2 — company data

Add Companies House profile/search/officer endpoints, provider credentials, rate handling and caching.

Phase 3 — public-data expansion

Add food hygiene, crime and flood endpoints.

Phase 4 — planning

Add curated England planning constraints with strong coverage/freshness metadata.

Phase 5 — commercial layer

Add Stripe/Cashier plans, quotas, dashboard usage charts and upgrade flow.

Phase 6 — launch hardening

Load testing, security review, source terms review, backup restore test, incident runbook, status page and production monitoring.

19. MVP acceptance criteria

The MVP is ready for public launch when:

- at least six endpoint families are live;
- every public endpoint is represented in OpenAPI documentation;
- all customer API requests require a valid key except explicitly public health/docs endpoints;
- quotas and burst limits are enforced;
- usage is metered accurately enough to reconcile a customer month;
- API keys can be created and revoked without support intervention;
- free and at least one paid plan work end-to-end;
- upstream timeouts do not exhaust application workers;
- source outages produce controlled errors or safe stale responses;
- secrets and API keys are not emitted into application logs;
- automated tests cover normal, invalid, rate-limited, cached and upstream-failure paths;
- backups and restore procedures have been tested;
- licensing/attribution has been reviewed for every launch source;
- dashboard, docs and API are usable on mobile and desktop;
- status monitoring exists independently of the application itself.

20. Initial KPI set

Do not over-instrument the first release. Track:

- registered users;
- users who make their first successful API request;
- time from registration to first successful request;
- monthly active API keys;
- total API requests;
- cache hit ratio;
- p50/p95 response latency by endpoint family;
- 4xx and 5xx rates;
- upstream error rate;
- free-to-paid conversion;
- monthly recurring revenue;
- quota-triggered upgrade conversions;
- churn/cancellations.

The strongest early product metric is likely **developers making a second API request on a later day**, because it indicates UKAPI.io has become part of an actual project rather than a one-off experiment.

21. Post-MVP opportunities

Potential additions after product-market validation:

- vehicle and MOT integrations where licensing/access is suitable;
- weather/location conveniences;

- schools and education datasets;
- NHS/service directories where reuse is appropriate;
- energy performance certificate/property datasets;
- election and constituency reference data;
- station/transport reference data;
- address lookup through a properly licensed commercial supplier;
- webhook notifications for changing source data;
- bulk/batch endpoints;
- downloadable snapshots;
- SDKs for PHP, C#, TypeScript and Python;
- MCP/tool integrations for agentic applications;
- organisation/team accounts;
- enterprise rate limits and contractual SLAs.

22. Recommendation

Build UKAPI.io as a **small Laravel monolith with strongly separated provider adapters**, not a microservice platform. Launch with breadth at the endpoint level but restraint at the infrastructure level. The differentiator should be developer experience: predictable contracts, excellent examples, sensible caching, low prices and a growing catalogue of useful UK-specific endpoints.

The product becomes defensible gradually as it accumulates normalized schemas, source knowledge, edge-case handling, usage data, SDKs and trust. None of those require a large team to begin.

References and current source notes

- Laravel 13 release notes: https://laravel.com/framework/docs/13.x/releases
- Laravel deployment documentation: https://laravel.com/framework/docs/deployment
- Companies House API: https://developer.company-information.service.gov.uk/get-started
- GOV.UK reuse / Bank Holidays API: https://www.gov.uk/help/reuse-govuk-content
- Police.uk API: https://data.police.uk/docs/
- Food Standards Agency FHRS API: https://api.ratings.food.gov.uk/help
- Planning Data API: https://www.planning.data.gov.uk/docs
- Environment Agency flood-monitoring API: https://environment.data.gov.uk/flood-monitoring/doc/reference
- Postcodes.io: https://postcodes.io/docs/overview/

Source capabilities and terms should be rechecked immediately before production launch. This report reflects public documentation reviewed on 23 September 2026.